Modelado y Programación: Reporte del proyecto 1

Martínez Méndez Ángel Antonio

March 27, 2026

Universidad Nacional Autónoma de México
Facultad de Ciencias
Profesor: Canek Peláez Valdés

Introducción

En este documento se discute el diseño y modelado del primer proyecto de la materia de Modelado y Programación de la carrera Ciencias de la Computación. Este se trata de un chat grupal el cual tiene diferentes funcionalidades como: mandar mensajes públicos, privados, cambiar de estatus, crear cuartos, invitar a otros usuarios a dichos cuartos, etc.

Iniciaremos mostrando algunas ideas que tuvimos cuando se nos presentó este proyecto. Luego, listamos las herramientas que usamos y por qué las elegimos y por último iremos repasando los problemas que se nos presentaron al momento de ir avanzando el proyecto.

Por último ofrecemos algunas conclusiones a las que llegamos una vez llegó la fecha límite de entrega del chat. Y también hacemos una breve reflexión sobre lo que hicimos bien y mal en la resolución de este programa.

1 Ideas para atacar al problema

En clase se nos recomendó no ir y enfrentar el problema yendo directo al código y tecleando como unos degenerados. Sino que, más bien, se nos invitó a reflexionar un poco sobre las partes que debería tener un chat grupal y cómo podríamos dividir el problema para atacar cada parte del mismo e ir construyendo de esta forma la solución.

Dada nuestra falta de experiencia en la creación de proyectos desde 0, no supimos ni cómo hacerle, porque básicamente el consejo iba más encaminado en entender conceptualmente el problema y luego pensar en cómo programarlo. Así la primera idea que tuvimos fue hacer un solo ejecutable que contuviera toda la solución. Sin duda toda nuestra abstracción fue de muy alto nivel y superficial, porque no descompusimos realmente bien el problema. Creímos que era sencillo hasta que notamos que este tipo de proyectos necesitan de un programa que sirva como mediador de la información (un servidor) y otro que transmita la información que se quiere mandar (un cliente).

Lo cual nos llevó otra vez al inicio, no lo notamos en un principio, pero prácticamente estamos usando el desarrollo de cascada (waterfall) para analizar el problema. El cual consta de al menos cinco partes: Requerimientos, Diseño, Desarrollo, Testeo, Mantenimiento. Se podría decir que ya queríamos avanzar a la parte de programación dejando el análisis del diseño con mucho que desear.

Los requerimientos solo trataban del protocolo (TCP), un tema que vamos a discutir más adelante, pero que tiene que ver con la parte que maneja la información del chat. Ahora bien, en nuestro segundo intento por analizar este problema sí logramos abarcar más a profundidad el diseño de nuestro programa. Como se mencionó antes, al menos necesita de dos ejecutables para poder funcionar; el servidor y el cliente. Y como no puede ser de otra forma, los analizaremos con Orientación a Objetos (POO).

1.1 El servidor

De manera abstracta necesitamos de un individuo que pueda recibir, transmitir y entender la información que se vaya recolectando a lo largo del programa. Antes habíamos dicho que era algo así como un mediador; lo cual nos parece acertado. Un servidor justamente tiene este tipo de comportamiento, maneja la información de los clientes que van conectando al mismo.

De manera intuitiva podemos entender al servidor como la lógica que maneja un videojuego en línea, este lleva el encuentro de todos los jugadores (clientes), los identifica de alguna manera y durante la ejecución del mismo va mostrando a los usuarios la información de la partida. Tomando esto como analogía, nuestro Servidor tiene que tener al menos una Estructura de Datos para ir guardando o registrando a los clientes, el puerto al que se va a conectar en nuestra computadora y métodos que vayan manejando el flujo de la información.

1.2 El Cliente

El Cliente es una estructura u objeto sencillo que va a ir alimentando al Servidor de información para que este haga algo. En particular; que diferentes usuarios se comuniquen entre sí. Por lo que nos pareció sensato usar una estructura auxiliar que el cliente va a manejar para que el usuario final que use el programa no se dé cuenta de toda la lógica que va a pasar detrás del Servidor y Cliente. De tal forma, que el Cliente usa un objeto Usuario que tiene datos como: nombre, estatus, cuartos e invitaciones. Adelantamos que los cuartos e invitaciones son una Estructura de Datos, pero lo veremos más adelante.

1.3 Bibliotecas

El Cliente y el Servidor no son programas aislados; están sumamente ligados. De manera que, necesitamos Objetos que modularicen y hagan la carga de trabajo más fácil para nuestros dos ejecutables. De manera que las bibliotecas cumplen específicamente este rol. Al menos en términos de Desarrollo de Software, las bibliotecas son pequeñas partes de código que contienen algoritmos que pueden ser reutilizados una y otra vez. Justo lo que necesitamos. Por ejemplo, para la transmisión de información necesitamos un protocolo. No podemos simplemente decirle al Servidor:" haz algo". Y esperar que justo haga lo que necesitamos. Por lo que tener una serie de reglas para que el flujo de la información sea entendible tanto para el Cliente como el Servidor, es buena idea.

Por otro lado, necesitamos un lenguaje que entiendan tanto Cliente como Servidor. Como en los requerimientos vamos a manejar el protocolo con formato JSON (JavaScript Object Notation), no esperamos que el Cliente tenga que escribir un JSON para comunicarse con otros usuarios. Por lo tanto, necesitamos tanto un objeto que maneje algún lenguaje que nos inventemos y un traductor que pase de nuestro lenguaje al protocolo en formato JSON. De esta manera, delegamos funciones específicas a clases específicas y logramos que sea mantenible.

Como extra, igualmente hicimos una clase verificador, esta podría existir como método en la clase Lenguaje, pero para dividir aún más las tareas, conviene realizar una estructura que verifique que lo que nos pase el usuario está bien hecho, lo que nos da paso al siguiente punto.

1.4 La entrada estándar y la línea de comandos

Aquí nos topamos con el primer problema en el desarrollo de esta aplicación. ¿Cómo el usuario tiene que instanciar los ejecutables desde la entrada de comandos? Y ¿qué tipo de comandos tiene que pasar para que el servidor le responda algo? Aquí nos pareció muy sensato tener una clase o tipo de datos abstracto que verifique toda la información que va a recibir el programa.

La idea sería recibir la información del usuario, verificar que el programa pueda entenderla, traducirla al formato de nuestro protocolo y de alguna manera envolverla y mandar al Servidor. En este punto pensamos

que sería mejor buscar una biblioteca que maneje el stdin/stdout para el programa que elijamos. Por lo que al menos esta parte estaba cubierta.

1.5 Estructura

Por último necesitamos una manera de organizar todos nuestros tipos de datos abstractos antes de ir a ensuciarnos las manos al programar. Por recomendación del profesor, nos fuimos directo a usar el patrón de diseño Modelo Vista Controlador. Una decisión de diseño plausible dado que hasta ahora solo hemos especificado la parte del Modelo y algo del Controlador. Por lo que decidimos que era buena idea usar este patrón para cada ejecutable de nuestro programa.

La parte de vista era sencilla, simplemente era el resultado de que el Servidor recibiera y transmitiera bien la información y que el Cliente logre comunicarse con otros Clientes. La parte del Modelo mayormente ya la hemos especificado con todo lo antes dicho. Claro que falta algunas partes que nos gustaría entrar más en detalle, pero lo haremos más adelante. Y por supuesto, el Controlador tiene que ver con la entrada que da el usuario al programa, este se encarga de llamar a alguna parte del Modelo para manejar los datos y estos se muestran en la salida.

2 Herramientas utilizadas

Siguiendo la idea de que al menos tenemos que entender el 80% del problema para empezar a implementar la solución. Es ahora cuando nos lanzamos de lleno y exponemos las herramientas que usamos. Así como los subproblemas que aparecieron en esta parte del desarrollo.

2.1 El Lenguaje de programación

Inspirados por la propuesta que nos hizo el profesor de utilizar un nuevo lenguaje en el que no estuviéramos familiarizados; que en nuestro caso es Java. Decidimos utilizar como lenguaje de programación Rust. Esta decisión fue tomada porque es un lenguaje que no se limita a ser únicamente un lenguaje orientado a objetos e imperativo como Java, sino que es multiparadigma. Lo cual puede llegar a ser provechoso en algunas instancias de la resolución del problema. Sin embargo, como es más sencillo, al menos para la resolución de nuestro programa, trabajar con Orientación a Objetos; nos limitamos a usarlo de esta manera.

Por otro lado, al ser un lenguaje construido en C, nos pareció buena idea hacer el proyecto en este lenguaje como un acercamiento a la programación de bajo nivel, sin las desventajas de poder compilar nuestro programa y correr el riesgo de tener fugas de memoria. Esto lo evitamos, debido a que Rust cuenta con un Compilador muy robusto y amable con el programador. Que en tiempo de compilación, usa conceptos como el de **OwnerShip**, que en cierta forma te obligan a programar cuidando la memoria del programa y posibles errores lógicos que podría tener.

La idea era buena, solo que el tiempo que nos tomó y sigue tomando aprender bien este lenguaje, menguó en cierta medida el desarrollo del programa.

2.2 GitHub, Integración Continua y Dockerización

Para tener un flujo de trabajo más organizado, controlado y documentado. Usamos la plataforma de GitHub para ir subiendo los cambios que hicimos en el desarrollo del programa. De esta manera también tuvimos

una especie de historial que contaba nuestros intentos de añadir funcionalidades al programa o ver por qué no convenían

Como requerimiento, le pusimos Integración Continua (CI), que básicamente consiste en hacer que la misma plataforma de Git genere los test de nuestro programa. Esto con el fin para evitar subir código que no funciona y poder depurar más fácilmente el código. Además, se considera una buena práctica de programación hacer esto. Por lo que sin duda, es una herramienta que implementamos a lo largo del desarrollo de nuestro programa.

También se nos pedía utilizar Docker, una herramienta más enfocada a administradores de archivos que para programadores. Aunque, en realidad, resulta ser igualmente útil a pesar de lo anterior. Esto, porque básicamente se trata de un contenedor de una imagen de arranque para alguna computadora, solo que una versión mínima. En la cual, la idea era implementarla para que cualquiera que hiciera uso de nuestro programa no tuviera problemas al tratar de compilarlo y ejecutarlo. Esto porque antes tenían que usar algunos comandos de Docker; sencillos, para poder utilizar el programa y no tuvieran que lidiar con el problema de que no corra el mismo en su computadora. Decimos la idea era, porque no logramos implementarlo.

2.3 El protocolo

El protocolo son las reglas con las que un dispositivo puede transmitir y recibir información en orden, para no perder ni añadir información de más. Se podría decir que es el corazón de nuestro programa. Dado que de esta parte depende de qué tipo de información va a aceptar y rechazar. Y qué respuestas debe de generar y enviar. En este caso, usando el Cliente y el Servidor.

Estas reglas son prácticamente la especificación de todo el programa. Ya que nos indican la funcionalidad completa del programa y las situaciones con las que debería lidiar. Como mencionamos antes, la idea es hacer un chat grupal y el Servidor es nuestro mediador en el problema. Por lo que aquí empezamos a agradecer el hecho de haber pensado en el diseño desde el inicio.

Hicimos una enumeración de todo el protocolo, el cual iba a ser utilizado tanto con el Cliente, como por el servidor. De aquí, cuando el Cliente quiera usar alguna funcionalidad, que puede ser: mandar mensajes públicos, privados, cambiar de estado, ver la lista de usuarios conectados, generar cuartos privados, invitar a otros usuarios a dichos cuartos. El Servidor va a ser el encargado de ir generando dichos posibles usos, verificando que el usuario no haga nada raro. A través de la clase Verificador, Lenguaje, Traductor. Este debe verificar que el usuario se identifique bien y generar tanto el Servidor, los sockets para comunicarse y generar los hilos de ejecución para que cada Cliente no abuse de la concurrencia de nuestro programa.

2.4 Implementando el protocolo

La idea para implementar casi todas las funcionalidades tenía que ver con usar Diccionarios, tanto para ir guardando a los Clientes que se iban conectando, como para guardar los cuartos que podían generar. Así como las invitaciones a otros usuarios que estos podían hacer. Como mencionamos arriba, no tuvimos tanto tiempo para terminar de implementar esto. Sin embargo, con el diseño a mano la única gran dificultad que tuvimos fue en el lenguaje de programación. Las demás opciones de uso que tenían los clientes, eran relativamente sencillas, una vez el Servidor le daba o envolvía a cada Cliente en un Hilo de ejecución y un Socket para poder comunicarse con cada uno.

Sin embargo, reiteramos que una vez haciendo la parte del modelado, notamos que se simplifica mucho la problemática a la que, en primera instancia, nos parecía muy complicada. Y es aquí, donde nos dimos cuenta

de la importancia de este curso. Lo que nos lleva a la siguiente sección.

Conclusiones

Aunque no logramos terminar de implementar el proyecto, aprendimos sobre la importancia de ser disciplinado y bondadoso con el uso del tiempo que se nos brinda para hacerlo. Además, de romper en parte más pequeñas el problema que se nos da, pues delegar a otras clases u estructuras diferentes procesos que requiere el programa lo vuelve más modular y sencillo de entender para poder hacerlo escalable a futuro.

También hacemos notar lo autodidacta que te vuelve esta carrera y, en particular, la resolución de proyectos de este tipo; desde cero. En el cual, se hace evidente la importancia del modelado en la interpretación de problemas que se pueden computar. Ya que de lo contrario, muy probablemente sin antes pensar el diseño nos habría sido imposible siquiera empezar a atacar el problema.

Hay una analogía que me agrada y noté que me ocurrió en el desarrollo de este proyecto, que dice que uno puede estar perdiéndose de admirar el bosque por centrarse en un árbol. En algunas partes del diseño del programa me pasó que tuve parálisis por sobre analizar en conjunto una parte del código que creí que ya estaba bien dividida. Lo cual, al no poder centrarme en otras partes del mismo, me hizo perder tiempo. Y ahora viéndolo desde otra perspectiva, es un problema que me gustaría pulir en posteriores proyectos.